

Privacy Leakage and Lightweight Defenses in Graph Neural Networks for Sensitive Data

Krish Sharma

Academy of Math, Science, and Applied Technology
Ed W. Clark High School
Las Vegas, NV, USA
krish.sharma@unlv.edu

Shaikh Arifuzzaman

Department of Computer Science
University of Nevada, Las Vegas
Las Vegas, NV, USA
shaikh.arifuzzaman@unlv.edu

Abstract—Although graph neural networks (GNNs) are widely used on sensitive data, high classification accuracy does not guarantee the model does not reveal the nodes used during training. In this paper, the authors investigate whether an adversary can learn the membership of the supervised training set from only the node classification probabilities. The authors propose a benchmark that varies the homophily and edge density of synthetic graphs and validates on Cora and Citeseer. For each of the models evaluated, including Graph Convolutional Networks (GCN), GraphSAGE, logistic regression, and multilayer perceptrons (MLPs), the authors compare the results of three black-box attacks on the graphs and evaluate the effectiveness of five lightweight model defenses. The results reveal that GNN model privacy leakage is structure- and model-dependent rather than universal. For the synthetic graphs, GCNs exhibit the highest area under the ROC curve (AUROC) for the attack results under adverse settings (0.59 for low homophily and sparse connectivity compared with ~ 0.51 – 0.53 for GraphSAGE and the feature-only models). However, the feature-only models reveal more membership information for the nodes in the Cora and Citeseer graphs when the features of the training nodes are highly separable (MLPs exhibit AUROC values of 0.77–0.83 compared with 0.55–0.63 for GCNs). The label smoothing training technique for GNNs can increase the success of the membership inference attacks. None of the evaluated lightweight defenses offer a universal improvement in the privacy of the trained models. Depending on the graph structure and GNN model, the privacy of the training membership must be measured directly for each deployment of the models.

Index Terms—graph neural networks, membership inference, privacy, homophily, graph density, lightweight defenses, calibration, shadow models

I. INTRODUCTION

Graph Neural Networks (GNNs) have recently been deployed to a variety of relational data in fields as diverse as healthcare, finance, and communications. For instance, patient data from a hospital network could be represented as a graph, and the nodes of this graph could be used to predict the outcome of a given patient based upon that patient’s features. While the model might accurately classify the outcome of each of the patients that were part of the training set, that accurate classification does not answer an important question: was it possible for an adversary to determine whether any of those patients were part of the training set based upon the predicted outcomes for each of those patients? This question is known as the membership inference attack (MIA) problem [1]. The

high accuracy of a model does not necessarily answer this question, as it is possible for a model to have high accuracy in its training set while having different prediction scores for the training set relative to the test set of patients [3]. Furthermore, due to the structure of GNNs, the answer to the membership inference attack question is not as simple as with traditional machine learning models. For instance, in the scenario of node classification, the graph as a whole is visible to the model during training and inference, but the labels for each of the nodes is used. The structure of the graph, such as the homophily and edge density within the graph, may have an impact upon the privacy of the members of that graph [5], [7].

To answer this question, we developed a benchmark that enables evaluation of the impact of these graph structure factors while isolating them from the effects of other factors that may be confounded by the structure of a real world graph. Six different regimes of synthetic graphs were constructed that differed in their homophily and edge density, and models were trained on each of the graphs using both GNNs and non-graph models. Furthermore, five different lightweight defenses were applied to the training of the models, and the utility and privacy of those models were evaluated using three different black-box attacks. All experiments were performed according to a YAML configuration file, and each experiment was performed five times with randomly chosen seeds. Four research questions guided the study: does the GNN leak more than a model that does not use the graph structure of the data (RQ1)? Does the homophily and edge density of the graph impact the leakage of information about its members (RQ2)? Do the lightweight defenses mitigate the information leakage (RQ3)? Is there a utility-privacy tradeoff for these models (RQ4)?

Contributions. We release a YAML-driven experiment framework that automates the sweep of datasets, models, defenses, and seeds. We compare GCN and GraphSAGE to feature-only baselines on both synthetic and citation datasets. We evaluate three attack families and five defenses with the same hyperparameters over five different seeds. Section II reviews the related work. Section III defines the threat model and evaluation metrics. Section IV describes the benchmarking process step by step. Section V presents the results to address each research question. Finally, Sections VI–VII interpret the

results and offer our conclusions.

II. BACKGROUND AND RELATED WORK

A. Membership inference on classical models

Membership inference in classical machine learning models aims to determine whether a given data point was part of the model’s training set, using only the model’s outputs [1]. Most existing methods train shadow models using data similar to the training data of the target model, then train a classifier that can recognize if the target model is classifying the given sample as a member of the training set. Many methods train these models without having to make assumptions about the model’s architecture, or the distribution of the training data [2]. Most theoretical work considers that one possible reason that a model will exhibit a membership inference advantage is that it memorized the training data samples, resulting in higher classification confidence with training samples than with similar non-training samples [3]. More recent developments suggest that evaluating the methods using a target false-positive rate instead of accuracy may be more appropriate to represent the model’s security [4]. We use the area under the receiver operating characteristic (AUROC) curve for the attack results as a scalar metric to evaluate the security of the model; this value is the probability that a randomly sampled training member will be classified as a training member by the attack model with a higher score than a randomly sampled nonmember, and takes a value between 0 and 1 (a value of 0.5 indicates random guessing).

B. Membership inference on graphs

The problem of determining whether a graph sample was part of the training set of a GNN has been studied [5]. Furthermore, while graph classification problems are distinct from node classification problems in that the examples are subgraphs rather than graphs [6], there have been surveys of the attacks that can be performed on GNNs, including membership inference [7], [29], [30]. These attacks are generally restricted to node-level attacks that are score-based and black-box in nature.

C. Architectures and lightweight defenses

Among the most common models for GNNs are graph convolutional networks (GCNs) [8]. GraphSAGE is a different model that samples neighbors of a node differently from GCNs [9]. DropEdge is another model that randomly removes edges of a graph during training to regularize the learning process [10]. Some defenses to membership inference that have been proposed are based on masking the model’s output, such as masking the model’s output according to its confidence in the predictions [11]. There are also differentially private versions of GNNs such as DP-GCNs [12], [19]–[21]. However, the majority of lightweight defenses that have been published are not differentially private; hence, the study of these methods is independent of differentially private methods.

D. Homophily and graph structure

One possible attribute of a graph that may relate to membership inference is the concept of homophily: the idea that graphs tend to have edges between nodes that have the same class label. In social networks, for instance, individuals with similar interests will have social network connections to each other. In contrast, low homophily results in connections between nodes of different classes. In models like GCNs, the propagation of information between nodes can act as a low-pass filter for information, removing information that differs between neighbors [8], [26]. Because membership inference considers the similarity of predictions between nodes, the structure of the graph in which nodes are coupled in training can lead to privacy issues: two different models can exhibit similar accuracy on the training data, but have different results in terms of classification confidence for training data versus nontraining data samples.

E. Research gap

Most related work suggests that GNNs can leak membership information. However, in a single real-world GNN example, there are multiple factors to consider: the homophily of the graph, the density of the graph, the separability of the features of the training data, and the GNN architecture. In experiments using the Cora dataset, for example, it is difficult to determine if the model leaks membership information due to the structure of the graph or the bag-of-words features of the training data. In our work, we consider different synthetic graphs with various levels of homophily and density but with the same dimensionality of features. We train various GNNs on these graphs and compare the performance of the models to feature-only baselines subjected to the same membership inference attacks, as well as compare the performance of various lightweight defenses against each other on these graphs.

III. PROBLEM SETUP

A. Threat model in plain terms

Suppose a node classifier is deployed in practice and returns a probability vector for each queried node. The adversary knows the label of the node under consideration and wants to know whether this label was used during the training of the model. The adversary has black-box access to the model; that is, the adversary can only observe the probability vectors returned by the classifier [5]. The adversary will be successful in their attempt if the classifier can distinguish between nodes that were used during training and nodes that were held out during training using a simple classifier.

B. Membership predicate and notation

Let $G = (V, E)$ be an undirected graph with n nodes, each having a d -dimensional feature vector and a label from the set $\{0, \dots, C - 1\}$. Let the training mask $\mathcal{T} \subseteq V$ be the set of nodes whose labels appear in the supervised loss, and let the held-out set $\mathcal{S} \subseteq V$ be the set of nodes whose labels are not in $\mathcal{T}$. The sets $\mathcal{T}$ and $\mathcal{S}$ are subsets of the nodes in G ; all nodes participate in the propagation of messages

TABLE I
PRIMARY NOTATION USED THROUGHOUT.

Symbol	Meaning
n, d, C	Nodes, feature dimension, classes
X, y	Feature matrix and label vector
$z_v, \hat{p}_v$	Logits and softmax probabilities at v
$\mathcal{T}, \mathcal{S}$	Supervised training-mask and held-out node sets
$\phi(\cdot)$	Four-dimensional attack feature map
$h(G), \rho$	Observed homophily, density fraction
ECE	Expected calibration error ($B=10$ bins)
$\alpha, \tau_{de}, \tau_{es}$	Smoothing, DropEdge, sparsification rates

during training. Thus, membership in $\mathcal{T}$ or $\mathcal{S}$ is a problem in transductive semi-supervised learning. Given the graph G , the feature matrix X , the label vector y , and the training and held-out sets $\mathcal{T}$ and $\mathcal{S}$, the goal is to determine if a given query node v is a member of $\mathcal{T}$ or $\mathcal{S}$, using only the softmax probability vector $\hat{p}_v$ and the label y_v of that node. Table I summarizes the notation used throughout the paper. The success of the attacks is summarized by the attack AUROC. An AUROC of 0.5 means that the attack has no advantage in determining whether a given node is a member of the training set $\mathcal{T}$ or the held-out set $\mathcal{S}$; an AUROC closer to 1.0 indicates a higher success rate in distinguishing between members and non-members. Thus, a higher attack AUROC value indicates higher privacy risk; this is the opposite of the utility metrics, where higher is better. Consequently, both utility and privacy metrics must be jointly considered in the evaluation of a model.

Remark 1. *The evaluations described in this paper are performed in the transductive setting, wherein the training and held-out sets are subsets of the same graph G . Consequently, the privacy attack setting is slightly weaker than if the model were hidden from the existence of entire sets of nodes in the graph; however, the privacy of members and non-members may still differ within the graph G due to the message-passing process. Consequently, the attacks described in this paper do not consider white-box models and attacks on model embeddings or gradients, structure-modification attacks, or federated machine learning models [7]. While these attack settings are important to consider in evaluating privacy in deep learning models, they are beyond the scope of this paper.*

C. Evaluation metrics

For each experiment, three families of metrics are reported. For each value of the utility metrics (defined below), the value is reported for both the training set and the held-out set of nodes. The utility metrics evaluate whether the membership attack impacts the utility of the model; higher values are better. For the privacy metrics, a higher attack AUROC indicates higher membership distinguishability, indicating that the model leaks information about the membership of the nodes in the training and held-out sets. Thus, for the privacy metrics, lower is better. The metrics reported in this paper are as follows:

Utility metrics. The utility of the model is evaluated using accuracy, macro F1, and macro classification AUROC on the

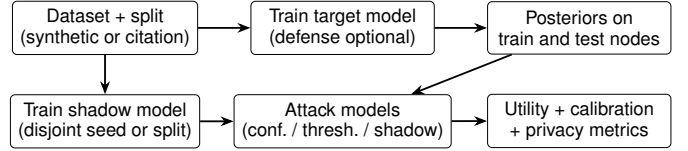


Fig. 1. End-to-end benchmark: target training, posterior extraction, shadow training, attack evaluation, and reporting.

TABLE II
FACTORS IN THE CONFIGURATION-DRIVEN EXPERIMENT GRID (YAML).
CARDINALITY IS THE CARTESIAN PRODUCT SUBJECT TO BASELINE
CONSTRAINTS (FEATURE-ONLY MODELS ONLY TRAIN UNDER NO
DEFENSE).

Factor	Levels
Synthetic regimes	6
Citation graphs	2 (Cora, Citeseer)
Model families	4 (LogReg, MLP, GCN, GraphSAGE)
GNN defenses	6 (none + five mechanisms)
Baseline defenses	1 (none)
Seeds $ \mathcal{S} $	5

held-out set of nodes. A higher value of accuracy means that fewer nodes are incorrectly classified by the model. Similarly, higher F1 scores indicate better performance by the model. Higher AUROC values mean that the model has better distinguishing capabilities between the different classes of labels.

Privacy metrics. The primary privacy metric is the confidence-attack AUROC, defined in Section IV-D. Additionally, a threshold attack and a shadow-model attack are also performed to provide additional evaluation of the privacy of the model.

IV. BENCHMARK DESIGN AND METHODOLOGY

A. Overview

Fig. 1 illustrates the end-to-end process followed for each experiment. Briefly, a graph is loaded or generated. A target model is trained on the graph (possibly with the use of a defense strategy). Posteriors are extracted from the model's predictions for the training and test nodes. A shadow model is trained on the training data. Finally, the membership models are evaluated for various metrics. Table II lists the factorial design of the experiment as implemented within the file `experiment_config_paper.yaml`. Each subsection that follows describes each of the components of the experiments in the order in which they are executed for any given experiment.

B. Datasets

Many properties of real world graphs are simultaneously present. These properties include homophily, density, the types of features of the nodes, and the structure of the communities within those graphs. Thus, we begin our experiments with synthetic graphs in which we can control homophily and density independently, and then extend to citation graph benchmarks.

Synthetic generator. Each synthetic graph has $n=400$ nodes, $d=50$ features, and $C=5$ classes. Labels are sampled

TABLE III
SYNTHETIC GRAPH GENERATOR: FIXED AND VARIED FACTORS.

Factor	Levels	Role
Homophily target	low (≈ 0.3), high (≈ 0.8)	Same-label edge fraction
Density ρ	0.005, 0.015, 0.04	Fraction of possible edges
Nodes n	400	Fixed order
Features d	50	Input dimensionality
Classes C	5	Label cardinality
Split	50% train / 50% test	Resampled per seed

TABLE IV
APPROXIMATE UNDIRECTED EDGE BUDGET BY DENSITY AT $n = 400$.

Density label	ρ	Edges (approx.)
Sparse	0.005	~ 400
Medium	0.015	~ 1200
Dense	0.040	~ 3200

uniformly. Features are drawn from class-specific Gaussian clusters: each class has a center μ_c , and node features are $x_v = \mu_{y_v} + \epsilon_v$ with small noise ϵ_v . This yields separable but noisy features, similar in spirit to a simple planted-cluster model.

Edges are generated to reach the two targets simultaneously. The density of the generated graphs is controlled by the parameter ρ , which takes on the values of $\{0.005, 0.015, 0.04\}$. Such a parameter controls the fraction of possible edges that are created between nodes in the graph; at a constant number of nodes $n = 400$, this means that the number of edges in the graphs ranges from approximately 400 (sparse) to 3200 (dense). The homophily target $h_{\text{tgt}} \in \{0.3, 0.8\}$ controls the fraction of edges that connect nodes of the same label to nodes of the same label. After the graph is generated, the measured homophily for that instance of the generated graph is $h(G) = |E|^{-1} \sum_{(i,j) \in E} \mathbb{1}\{y_i = y_j\}$. This value may not exactly equal the targeted value due to the rejection sampling process that was applied to generate the graph. A combination of the crossing homophily and density regimes creates the six named regimes in Table V. Fig. 2 presents the different stages of the process that was described above. For each randomly selected seed, the graph is resampled with respect to the node labels, features, graph edges, and the training-test split of the nodes. The pseudocode for resampling the edges is listed in Appendix A.

Citation benchmarks. For the experiment on the Cora and Citeseer graphs (Table VI), we use the same model, attack, and defense methods that were used for the synthetic graphs. However, for those graphs, we resample the training and test splits of the nodes to ensure that each randomly selected seed has a randomly selected training and test split. This ensures that the same membership predicate is used as for the synthetic graphs: nodes are split 50/50 between training and test splits, and all nodes are members of the graph during the training process.

C. Predictive models

We compare two models based on graph networks with two baselines that do not consider the adjacency matrix at all. The goal is to remove any possible influence of the GNN on the results to ensure that any extra leakage that occurs beyond the feature-only models is indeed due to the GNN.

Logistic regression is a model that applies a softmax classifier directly to the features of each node in the network, x_v . This model will help to determine if the features alone introduce any leakage of membership information.

Multilayer perceptrons (MLP) add hidden layers of nodes on top of the features but ignore the adjacency matrix. Adding hidden layers allows the model to learn more complex relationships between the features.

Graph convolutional network (GCN) takes into account both the features of each node and those of its neighbors in the graph via the adjacency matrix [8]. The graph is transformed using $\tilde{A} = D^{-1/2}AD^{-1/2}$ [8]. Each layer of the graph convolutional network calculates [8]

$$H^{(1)} = \sigma(\tilde{A}XW^{(0)}), \quad Z = \tilde{A}H^{(1)}W^{(1)}, \quad (1)$$

The activation function is the rectified linear unit (ReLU) activation function and dropout with rate 0.5 is included in the calculation of $H^{(1)}$. The weight matrices are $W^{(0)} \in \mathbb{R}^{d \times 64}$, $W^{(1)} \in \mathbb{R}^{64 \times C}$. Each of these calculations is performed to mix the features of each node with the features of its neighbors to allow for the influence of the graph structure to be included in the predictive model.

GraphSAGE is another model that uses two layers but uses mean pooling to aggregate information from the neighbors of each node rather than using the spectral graph convolution used by GCNs [9]. This model has been shown to be more robust when the neighboring nodes have different classes than each other. All of the models are trained using the Adam optimizer to minimize the cross-entropy loss only on the training set $\mathcal{T}$.

D. Membership inference attacks

To train the membership inference attack models, we use the four scalars of the prediction vector, $\hat{p}_v$, and the true label of each data point, y_v [2]. These four scalars for each node v are defined in equations (2) and (3):

$$\phi_1 = \max_c \hat{p}_{v,c}, \quad \phi_2 = \hat{p}_{v,y_v}, \quad (2)$$

$$\phi_3 = -\sum_{c=1}^C \hat{p}_{v,c} \log(\hat{p}_{v,c} + \varepsilon), \quad \phi_4 = -(1 - \phi_1) \log(\phi_1 + \varepsilon). \quad (3)$$

Each of these features represents a different value of the model and prediction of each node in the graph. ϕ_1 indicates the confidence of the model in its best prediction for the node's class. ϕ_2 is the confidence that the model has in the true label of the node. Often, the training data for the model contains many instances of the true label of each training point so the model is very confident in its prediction of the label of those nodes. ϕ_3 is the entropy of the prediction vector. This scalar

TABLE V
SYNTHETIC EVALUATION REGIMES AT $n=400$ (UNDIRECTED). OBSERVED $h(G)$ AND $|E|$ VARY SLIGHTLY PER SEED.

Dataset ID	Homophily band	ρ	$ E $ (approx.)	Purpose
synthetic_high_sparse	high (~ 0.8)	0.005	~ 400	Sparse assortative
synthetic_high_medium	high	0.015	~ 1200	Medium assortative
synthetic_high_dense	high	0.040	~ 3200	Dense assortative
synthetic_low_sparse	low (~ 0.3)	0.005	~ 400	Sparse heterophilic
synthetic_low_medium	low	0.015	~ 1200	Medium heterophilic
synthetic_low_dense	low	0.040	~ 3200	Dense heterophilic

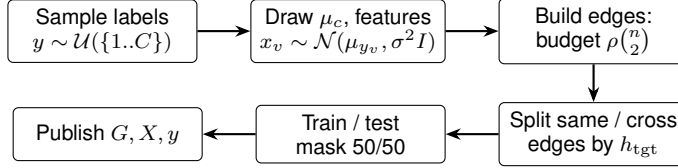


Fig. 2. Controlled synthetic pipeline: labels drive feature clusters; edges satisfy density ρ and homophily target h_{tgt} ; masks split nodes for semi-supervised training.

TABLE VI
PLANETOID CITATION BENCHMARKS.

Dataset	Nodes	Edges	Features	Classes
Cora	2,708	5,429	1,433	7
Citeseer	3,327	4,732	3,703	6

represents the distribution of the confidence of the model’s predictions for each of the classes. If the model is confident in its prediction of the label of each node, then ϕ_3 will be small. When ϕ_3 is large, it means that the model spreads its confidence equally between all classes. ϕ_4 returns some measure of the confidence of the model in the difference between its best predicted label and the true label of the node. The four scalars in equations (2) and (3) are referred to collectively as the ϕ -vector of each node. These ϕ -vectors represent the information that will be fed into the membership inference attack models described in the following section.

Confidence attack. A model with ϕ -vectors from all the nodes in the training set $\mathcal{T}$ and the test set $\mathcal{S}$ is created. These ϕ -vectors are split into two separate groups of 50% each. Using the ϕ -vectors from one group, we train a logistic regression classifier model. The trained model is used to predict the membership status of the nodes in the other group. This model’s area under the receiver operating characteristic curve (AUROC) is reported as a metric of the success of the membership inference attack.

Threshold attack. This model is the simplest attack model; it uses only the value of ϕ_2 for each node. This value is compared directly to the median of the value of the true label of all the nodes in the network. If the value of the true label of a node is lower than the median of all the nodes in the network, it is classified as an ordinary node that does not belong to the model’s training set.

Shadow-model attack. An attacker that uses the shadow-model attack trains a shadow model with the same architecture

as the target model and with the same privacy-preserving defenses but with data that is independent of the training data for the target model [1]. The shadow model’s ϕ -vectors are learned and used to predict the membership status of the target model’s training data points. If the shadow model can successfully predict the membership status of these nodes, it suggests that the membership signal is not specific to the training data of the target model.

E. Lightweight defenses

Besides using differential privacy to train the graph neural networks, there are also other techniques that can be used to protect the membership information in the graph. The five presented defenses are among the most commonly used by practitioners without the need to implement differentially private infrastructure. All of the defenses are applied only to the graph neural networks, while the baselines are trained using vanilla cross-entropy loss during training. All hyperparameters are held fixed across the grid (Table VII).

None (baseline). The baseline training uses cross-entropy loss with Adam optimizer on the training data $\mathcal{T}$.

DropEdge [10] randomly deletes 30% of the edges in the graph at each epoch ($r_{de} = 0.3$). The full graph is restored when making predictions with the model.

Label smoothing is applied to the training data. Instead of minimizing the cross-entropy loss between the true labels and the model predictions, smoothed labels are used, where the true label is replaced with a distribution over all classes (and each class is assigned a probability of $1/C$). The smoothing parameter α is set to 0.1. While this type of smoothing is often used to regularize vision models to improve the calibration of their predicted softmax outputs, we are curious whether this type of regularization has any effect on reducing the membership information leakage from the graphs.

Early stopping is used during training. Training is halted if the training loss does not drop by a small value ($\delta = 10^{-4}$)

TABLE VII
DEFENSE HYPERPARAMETERS (FIXED ACROSS THE GRID).

Defense	Value
DropEdge	$r_{de} = 0.3$
Label smoothing	$\alpha = 0.1$
Early stopping	patience $p = 15$, $\delta = 10^{-4}$
Confidence masking	top- k , $k = 2$
Edge sparsification	$r_{es} = 0.2$

Algorithm 1 Single trial (dataset, model, defense, seed)

Require: Graph family, model class, defense configuration, seed s

- 1: Instantiate or load data; apply train-test split from s
- 2: Train target model with defense on training mask
- 3: Compute posteriors on training and test nodes
- 4: Train shadow model on independent split or regenerated graph
- 5: Compute ϕ -features; run confidence, threshold, shadow attacks
- 6: Evaluate accuracy, F1, AUROC, ECE on test mask
- 7: **return** Metric vector (stored per row)

after training for $p = 15$ epochs. This is intended to prevent the model from learning by heart all of the training examples.

Confidence masking is applied during inference to the model predictions. The predictions for all classes except the $k = 2$ most likely classes are set to zero. This mimics API functions that may not return class labels with low confidence. The same masking is applied to the shadow models.

Edge sparsification deletes each edge in the graph with probability $r_{es} = 0.2$ prior to training and trains the model using this sparse graph for both training and inference. Thus, this method both randomly deletes edges at each training iteration (like DropEdge), but it also permanently deletes edges from the graph before training begins.

F. Experimental protocol

All models train with Adam ($\eta=0.01$, weight decay 5×10^{-4}) for up to $T=50$ epochs on CPU. Seeds $\mathcal{S} = \{42, 123, 456, 789, 1024\}$ control synthetic graph generation, train-test splits, and attacker subsplits. The file `experiment_config_paper.yaml` declares the full grid: Cora, Citeseer, six synthetics, four models, three attacks, five defense mechanisms plus a no-defense baseline, and five seeds.

For each GNN defense d , we compare confidence-attack AUROC against the no-defense baseline using paired differences across seeds and exploratory two-sided t -tests. We also report bootstrap confidence intervals over seeds. Because we test many regime-defense pairs, p -values should be read as exploratory indicators rather than definitive claims. Algorithm 1 summarizes one trial; Appendix B describes the software stack and reproducibility workflow.

V. RESULTS

We organize findings by RQ1–RQ4 so each subsection answers one question, presents supporting figures and tables,

TABLE VIII
MEAN CONFIDENCE-ATTACK AUROC WITHOUT DEFENSE ($|\mathcal{S}|=5$), SYNTHETIC REGIMES.

Setting	LogReg	MLP	GCN	SAGE
High homophily, dense	0.485	0.536	0.494	0.509
High homophily, sparse	0.532	0.504	0.528	0.539
Low homophily, medium	0.535	0.561	0.563	0.534
Low homophily, sparse	0.508	0.506	0.592	0.520

and closes with an explicit takeaway. Unless stated otherwise, summaries aggregate five random seeds; confidence-attack AUROC is the primary privacy metric. Higher attack AUROC means higher privacy risk.

A. RQ1: GNN versus feature-only vulnerability

Claim. Graph neural networks are not more vulnerable than feature-only models.

Synthetic graphs. Fig. 3 and Table VIII present results of four different model families on six different types of synthetic graphs with no defenses included. In settings with a high degree of homophily (nodes with similar features tend to connect to one another), all the models have an attack AUROC that clusters around 0.5. In settings with low homophily and sparse connectivity, however, the GCN model achieves an AUROC of ~ 0.59 , while the other three model types achieve only ~ 0.51 to 0.53. Furthermore, as presented in Fig. 4, the heatmap for the same experiments shows the GCN model’s AUROC brightening to higher values in the low-homophily and sparse graphs. Thus, the GCN appears to leak more information about membership in such regimes.

Citation benchmarks. While synthetic graphs allow for isolation of structural features of the graphs, real-world graphs also include features of nodes themselves. As presented in Fig. 5 and in the numbers below the figures, for the Planetoid citation graphs, the results of the attack without any defensive mechanisms show the feature-only models to have higher AUROC values than the GNN models; the logistic regression model reaches an AUROC of 0.735, the MLP model achieves 0.772, the GCN model 0.554, and the GraphSAGE model 0.635. Furthermore, the same trends appear on the Citeseer graphs: the logistic regression model reaches 0.730, the MLP model reaches 0.830, while the GNNs reach 0.632 (GCN) and 0.720 (GraphSAGE). The features of the nodes in the Planetoid graphs are highly separable on average; thus, models that fit those features well (like the MLP) leak more membership information than models that smooth node features over their neighborhoods (like the GCN).

Takeaway (RQ1): There is no general rule that GNNs leak more membership information than feature-only models. On synthetic graphs, the GCN model can leak more information in settings with low homophily and sparsity; on the citation benchmarks, however, the feature-only models leak more information overall.

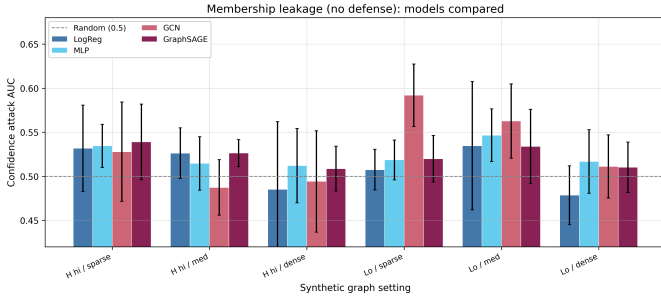


Fig. 3. Attack AUROC without defense across model families (synthetic regimes).

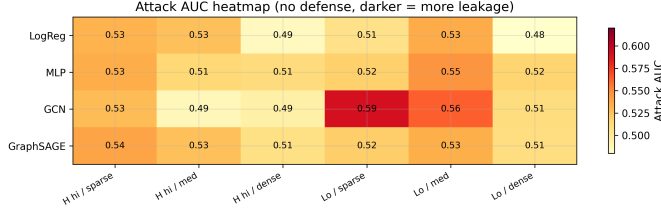


Fig. 4. No-defense attack AUROC heatmap across models and synthetic regimes.

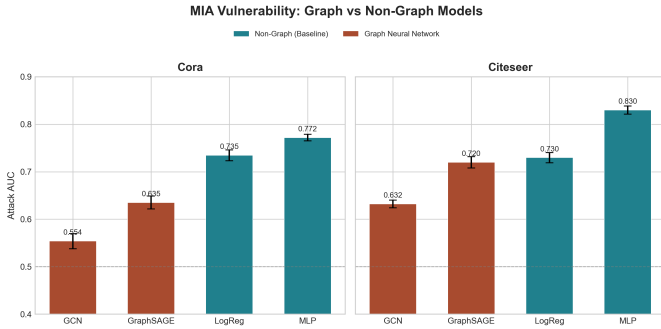


Fig. 5. Citation benchmark comparison across model families.

B. RQ2: Homophily and density

Claim. For GCN, attack AUROC tends to increase as homophily decreases and as density ρ decreases. GraphSAGE often achieves strong classification accuracy while attack AUROC remains near chance.

Fig. 6 plots the AUROC of the GCN attack across the six synthetic regimes. As expected, within a fixed density regime, the homophily decreases from right to left, and the leakage increases. Furthermore, within the low-homophily regime, the sparse graphs have a higher AUROC than the dense graphs. Fig. 7 depicts the relationship between homophily and AUROC for several different model-defense combinations. Fig. 8 presents the effect of graph density for different levels of homophily.

GraphSAGE yields surprisingly different results from the GCN: it often outperforms the GCN on accuracy but has a confidence-attack AUROC close to 0.5 even in heterophilic settings. Such behavior has implications for the deployment of GraphSAGE: a site administrator who relies only on accuracy to gauge the safety of a model could incorrectly assume

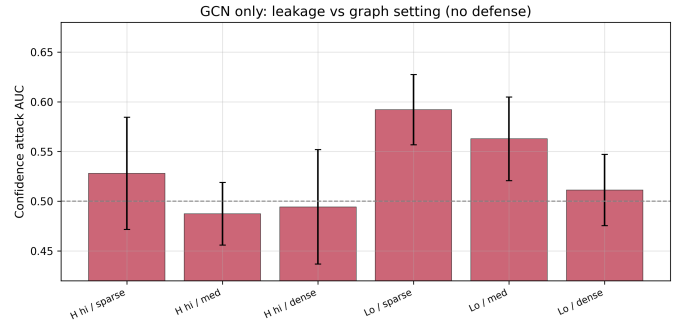


Fig. 6. GCN without defense across synthetic regimes.

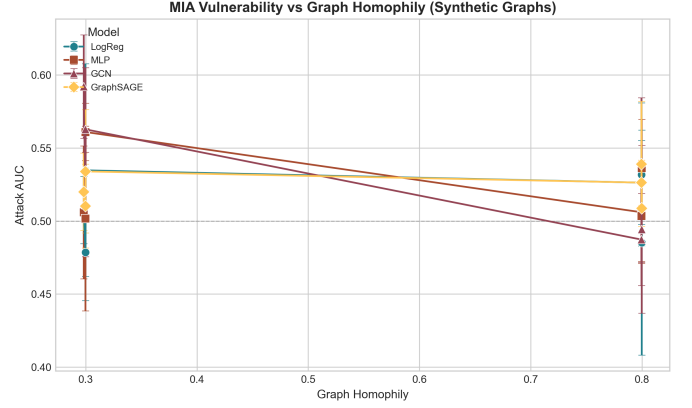


Fig. 7. Attack AUROC versus measured homophily in synthetic regimes.

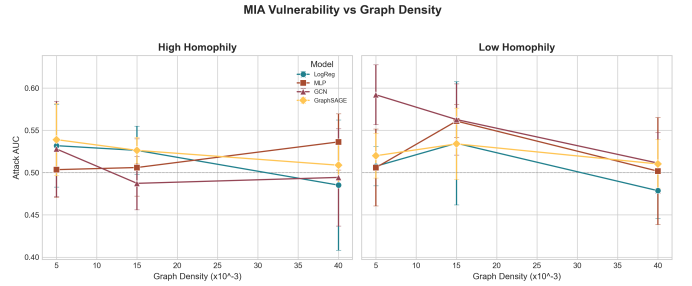


Fig. 8. Density-wise leakage trends across homophily regimes.

that GraphSAGE is either safe or unsafe with respect to membership distinguishability.

Takeaway (RQ2): The structure of the graph has an impact on the privacy risk of the GCN model in a way that is both predictable and generalizable across regimes; accuracy and membership distinguishability are related metrics, but they are not interchangeable.

C. RQ3: Lightweight defenses

While label smoothing is often introduced to reduce the overconfidence of models, one might expect it to hide the membership of training examples. Our experiments show the opposite in several cells. In Table IX, we report the results of applying label smoothing to GraphSAGE on low-homophily sparse graphs (+0.094 AUROC, $p = 0.0421$), Cora with a GCN (+0.039, $p = 0.0039$), and Citeseer with

TABLE IX
ILLUSTRATIVE PAIRED-TEST HIGHLIGHTS FOR CONFIDENCE-ATTACK AUROC. POSITIVE Δ MEANS HIGHER LEAKAGE; p -VALUES ARE EXPLORATORY.

Setting / model	Defense	Δ AUC	p
Low-medium / GCN	Edge sparsif.	+0.001	0.9684
Low-sparse / SAGE	Label smooth.	+0.094	0.0421
Cora / GCN	Label smooth.	+0.039	0.0039
Citeseer / SAGE	Label smooth.	+0.067	3.1×10^{-6}

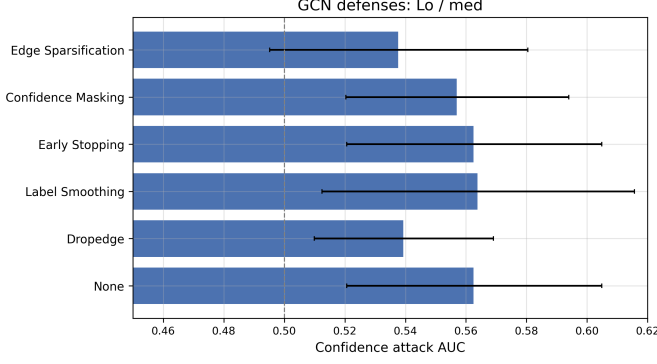


Fig. 9. GCN defenses on low-homophily medium-density graphs.

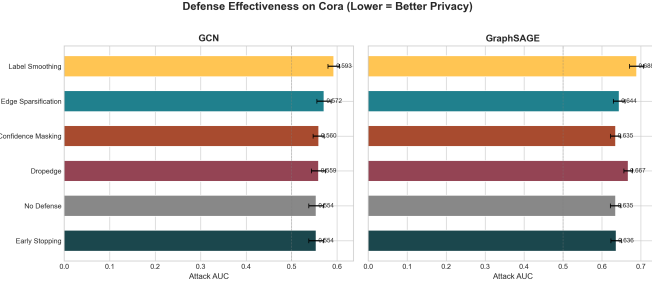


Fig. 10. Defense effectiveness ranking for citation benchmarks.

a GraphSAGE model ($+0.067$, $p = 3.1 \times 10^{-6}$). In Fig. 11, we show label-smoothed models tend to be warmer than their no-defense counterparts. Moreover, label smoothing also makes the models more susceptible to calibration error. In the remainder of the paper, we investigate a few additional defenses: edge sparsification. In low-homophily medium-density graphs, GCN with sparsification achieves the same attack AUROC as when edges are not sparsified (0.563 vs. 0.563, $p = 0.9684$). Similarly, permanently thinning the edges in a graph neither significantly helps nor hurts a model trained with the GraphSAGE algorithm. Other defenses like DropEdge, early stopping, and confidence masking exhibit mixed results (Fig. 9 and Fig. 10). For effective defenses, auditing a model with attacks specifically on that target graph and model is required. Label smoothing, therefore, should not be applied to models as a means of providing privacy by default.

D. RQ4: Utility–privacy tradeoffs

In general, the best utility–privacy tradeoffs will exist for models that exhibit a high level of generalization, such as models that are trained with high homophily values or with



Fig. 11. Comprehensive attack AUROC heatmap across datasets, models, and defenses.

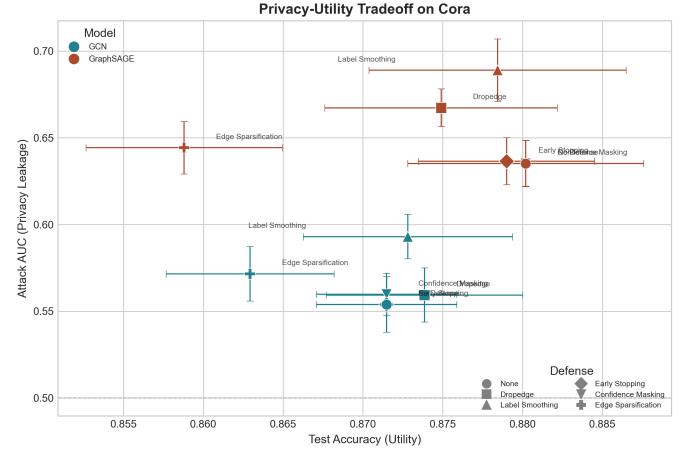


Fig. 12. Privacy–utility scatter across GNN defenses.

GraphSAGE in cases of heterophilic graphs. The worst utility–privacy tradeoffs will exist for any models that have an adverse structure in their graphs (such as using a GCN with low-homophily sparse graphs) combined with any defenses that increase the privacy leakage of the model. In Fig. 12, we plot the accuracy of the models against their AUROC scores for graph attacks for each of the defenses. For an ideal privacy-preserving model, the points would lie along the upper-left region of the plots, where accuracy is high and the AUROC for attacks is low. For the label-smoothed models, however, the points shift rightward in the plots, indicating that accuracy is maintained but the models exhibit higher leakage.

VI. DISCUSSION

A. Why these patterns occur

High homophily allows a GCN to produce similar posteriors for both training and held-out nodes during training. Thus, there is no signal for an attacker leveraging a score-based attack method to determine the membership status of a held-out node. With lower homophily, the GCN might produce different outputs for training nodes than for held-out nodes;

the GCN’s parameters are updated during training using the labels of training nodes but not for held-out nodes. This leads to potential exploitation by an attacker using ϕ -features [3], [5], [25], [26]. For citation graphs, the features that are among the most separable are the bag-of-words features. Thus, models relying solely on linear and MLP models memorize these features well but exhibit smoothing with GCNs that average the bag-of-words features.

B. Defense pitfalls

Applying label smoothing to a model redistributes the probabilities of model predictions away from the true label. However, there is no guarantee that the distribution of probabilities for members and non-members will become indistinguishable from one another. In several cells, label smoothing led to an increase in both ECE and attack success (AUROC) [11]. Furthermore, approaches like edge sparsification and DropEdge affect the information flow within a graph model. However, their effectiveness as a privacy-preserving method relies upon whether the GCN model was over-relying upon specific edges to memorize the training labels. There is no calibration or topology alteration technique that can be considered private without evaluating its effectiveness against score-based black-box attacks on the specific graph upon which the model is to be deployed.

C. Limitations and external validity

The synthetic graphs used contain only two homophily bands and blob features. Other structures of interest in graph analysis, such as communities or the continuous dimension of heterophily, are not represented in these graphs. The findings of this investigation are limited to the types of graphs that can be generated by our model and do not represent any general laws regarding all possible graphs. Additionally, the citation benchmark graphs are not exclusively synthetic but do include real-world graphs. Thus, the validity of this experiment is confounded by the fact that the features of these graphs are often different from those used in the synthetic graph experiments. Furthermore, only score-based black-box attacks are evaluated. An analysis of label-only [11] or likelihood-ratio tests [4] would have revealed different results. Five different seeds were used in the experiments. The statistical significance of the results was evaluated using the paired t -test, and the p -values obtained are exploratory in nature. The lightweight defenses evaluated in this study are not a substitute for applying differentially private training techniques when strong privacy guarantees are required [12].

D. Practitioner guidelines

Based upon these findings, graph model deployers should: (i) make certain that the predicate of the membership definition is carefully defined prior to evaluating the privacy of the model; (ii) report the confidence and ECE of a model’s predictions in addition to its accuracy; (iii) avoid treating label smoothing as a method of ensuring the privacy of a model without testing the model against score-based black-box

attacks on the specific graph upon which the model will be deployed; (iv) treat edge sparsification and similar techniques as a hypothesis to test rather than a guarantee of membership privacy; and (v) ensure that their model is robust against attacks stronger than linear ϕ -attacks if their concern regarding the privacy of the model is to be strong [4].

VII. CONCLUSION

In this work, we have investigated whether deep graph neural networks (GCNs) leak membership information of supervised training nodes and whether simple defenses against membership queries reduce such information leakage. Our findings indicate that information leakage is setting-dependent. GCNs tend to leak membership information most under low homophily and sparse synthetic graphs. Furthermore, models that do not rely upon graph messages, such as those using only bag-of-words features, tend to leak more under citation graphs than do GCNs. Label smoothing, a popular technique to improve generalization, often increases the information leakage of the models. Thus, there is no guarantee that applying such smoothing will protect the privacy of the training data. The notion of privacy for any model must be measured on the specific graph, model, and defense that are to be used when deploying the model. We have also released a configuration-driven codebase that enables others to easily reproduce the experiments reported herein. In future work, it will be interesting to extend this work to use a larger budget for model seeds, adopt more strict attacks with lower false-positive rates, continue investigating the difficulty of features for model generalization, and consider differentially private deep learning methods to train GCNs [4], [12], [14], [22].

APPENDIX A

SYNTHETIC EDGE-SAMPLING PROCEDURE

Let $N_{\text{tgt}}(\rho) = \lfloor \rho n(n-1)/2 \rfloor$ be the number of undirected edges in the graph with density parameter ρ . With target homophily $h_{\text{tgt}} \in \{0.3, 0.8\}$, we sample $\lfloor h_{\text{tgt}} N_{\text{tgt}}(\rho) \rfloor$ proposed edges within each class and the remainder between classes. Candidate edges are sampled with the label constraints, deduplicated with a hash set, and the resulting pairs are stored in both directions in the PyTorch Geometric graph object. If the number of proposed edges is zero, at least one edge is added between two randomly chosen nodes in the graph. Due to the duplicate rejection step, the realized values for $h(G)$ and the density of the resulting graph may differ slightly from the targeted values; in all reported results, these realized quantities are presented for each seed of the synthetic graphs.

APPENDIX B

IMPLEMENTATION AND REPRODUCIBILITY

All experiments are conducted with Python 3, utilizing the PyTorch, PyTorch Geometric, scikit-learn, and pandas Python packages. The configuration file `experiment_config_paper.yaml` contains the settings for each experiment: the datasets to use, the models, defenses, and attacks to evaluate, the seeds for the

experiments, and the directories in which results should be written. Running the driver script (`run_final.py` with the `PRIVACYGNN_CONFIG` parameter set to the experiment configuration file) performs all experiments and writes the results to CSV files. The figure scripts within the repository read the CSV files and regenerate the plots presented in this paper. Randomness is introduced into the experiments at four different stages: splitting the datasets into training and test samples, sampling synthetic edges within the training set, applying the DropEdge defense masks during training, and splitting the training samples into two sets of equal size for training and testing the confidence attacker. Each experiment is run with the sequence of seeds $\{42, 123, 456, 789, 1024\}$ for these four steps. Each shadow model is trained with the same masking as the target model to simulate the API that the confidence attacker will face when attempting to attack the target. Each experiment is configured to run on a CPU; training the GCN models involves $\mathcal{O}(T|E|d_h)$ operations for T training epochs and hidden layer width $d_h = 64$, and training the attack models on a sample of $\mathcal{O}(n)$ four-dimensional feature vectors is negligible in comparison. The full set of experiments was performed on a CPU running $n = 400$ synthetic graphs.

APPENDIX C CONFIDENCE MASKING

Confidence masking of the graph neural network models involves discarding the logits for the k classes with the smallest logits (here $k = 2$), replacing their values with a large negative value (such as -100), and applying the softmax function to normalize the remaining logits. Attacks that rely upon the tail probabilities of the model’s logits distribution (such as entropy ϕ_3) will lose sensitivity to membership after masking; however, if the training examples have different concentrations of the model’s remaining probability mass upon the top two classes compared to the nodes held out for testing, the remaining entropies (ϕ_1 , ϕ_2 , and ϕ_4) may retain their sensitivity to model membership. The shadow models are masked in the same way as the target models to simulate the API that the confidence attacker will use against the target model.

CODE AVAILABILITY

The code that accompanies this paper is available at: <https://github.com/PrivacyAI-Lab/privacygnn>. The code includes the experiment configuration files, the Python models, the Python scripts to aggregate the experiments’ results in CSV format, and the matplotlib scripts to recreate the figures within this paper.

ACKNOWLEDGMENT

Research conducted with the Data Science Laboratory (DiSC), University of Nevada, Las Vegas.

REFERENCES

- [1] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, “Membership inference attacks against machine learning models,” in *Proc. IEEE Symp. Security Privacy (S&P)*, 2017, pp. 3–18.
- [2] A. Salem, Y. Zhang, M. Humbert, P. Berrang, M. Fritz, and M. Backes, “ML-Leaks: Model and data independent membership inference attacks and defenses on machine learning models,” in *Proc. Netw. Distrib. Syst. Security Symp. (NDSS)*, 2019.
- [3] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, “Privacy risk in machine learning: Analyzing the connection to overfitting,” in *Proc. IEEE Comput. Security Found. Symp. (CSF)*, 2018, pp. 268–282.
- [4] N. Carlini, S. Chien, M. Nasr, A. Song, T. Terzis, and F. Tramer, “Membership inference attacks from first principles,” in *Proc. IEEE Symp. Security Privacy (S&P)*, 2022, pp. 1897–1914.
- [5] I. E. Olatunji, W. Nejdl, and M. Khosla, “Membership inference attack on graph neural networks,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.
- [6] Y. Wu, D. Li, B. Chen, and L. Shen, “Adapting membership inference attacks to GNN for graph classification,” in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2021, pp. 1429–1434.
- [7] Y. Zhang, Y. Zhao, Z. Li, X. Cheng, Y. Wang, O. Kotevska, P. S. Yu, and T. Derr, “A survey on privacy in graph neural networks: Attacks, preservation, and applications,” *arXiv preprint arXiv:2308.16375*, 2023.
- [8] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.
- [9] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 1024–1034.
- [10] Y. Rong, W. Huang, T. Xu, and J. Huang, “DropEdge: Towards deep graph convolutional networks on node classification,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.
- [11] C. A. Choquette-Choo, F. Tramer, N. Carlini, and N. Papernot, “Label-only membership inference attacks,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 1964–1974.
- [12] S. Sajadmanesh, J. G. Castillo, A. Naseri, M. Maengel, J. Boedecker, and B. H. F. Gomes, “GAP: Differentially private graph neural networks with aggregation perturbation,” in *Proc. USENIX Security Symp.*, 2023, pp. 3229–3246.
- [13] M. Fredrikson, S. Jha, and T. Ristenpart, “Model inversion attacks that exploit confidence information and basic countermeasures,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2015, pp. 1322–1333.
- [14] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, “Stealing machine learning models via prediction APIs,” in *Proc. USENIX Security Symp.*, 2016, pp. 601–618.
- [15] C. Song, T. Ristenpart, and V. Shmatikov, “Machine learning models that remember too much,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2017, pp. 587–601.
- [16] M. Nasr, R. Shokri, and A. Houmansadr, “Comprehensive privacy analysis of deep learning: Stand-alone and federated learning under passive and active white-box inference attacks,” in *Proc. IEEE Symp. Security Privacy (S&P)*, 2019, pp. 739–753.
- [17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1321–1330.
- [18] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2005, pp. 625–632.
- [19] C. Dwork, “Differential privacy,” in *Proc. Int. Colloq. Automata, Languages and Programming (ICALP)*, 2006, pp. 1–12.
- [20] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, “Deep learning with differential privacy,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2016, pp. 308–318.
- [21] I. Mironov, “Rényi differential privacy,” in *Proc. IEEE Comput. Security Found. Symp. (CSF)*, 2017, pp. 263–275.
- [22] B. Jayaraman and D. Evans, “Evaluating differentially private machine learning in practice,” in *Proc. USENIX Security Symp.*, 2019, pp. 1895–1912.
- [23] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.

- [24] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.
- [25] H. Pei, B. Wei, K. C.-C. Chang, Y. Lei, and B. Yang, "Geom-GCN: Geometric graph convolutional networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.
- [26] J. Zhu, Y. Yan, L. Zhao, Y. Heimann, L. Akoglu, and D. Koutra, "Beyond homophily in graph neural networks: Current limitations and effective designs," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.
- [27] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open graph benchmark: Datasets for machine learning on graphs," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.
- [28] X. Zhu, Z. Ghahramani, and J. D. Lafferty, "Semi-supervised learning using Gaussian fields and harmonic functions," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2003.
- [29] V. Duddu, A. Boutet, and V. Shejwalkar, "Quantifying privacy leakage in graph embedding," in *Proc. ACM Int. Conf. Web Search Data Mining (WSDM)*, 2020, pp. 76–84.
- [30] M. Hay, G. Miklau, D. Jensen, P. Weis, and S. Srivastava, "Resisting structural re-identification in anonymized social networks," *Proc. VLDB Endowment*, vol. 1, no. 1, pp. 102–114, 2008.